

# tunan 端到端教程 — 投票系统

本教程不生成投票系统的真实代码。它把 tunan 工作流的每一步用一个具体例子（投票系统）走完，让你照着 sponsor 该说的话、claude 该有的响应，亲手把整套流程跑一遍。

教程分两轮：

1. **第一轮**：从对话生成 REQ → PRD → STORY → PLAN → TESTPLAN → worktree dev → PR 自驱动循环 → merge → verify
2. **第二轮**：预设一个 bug（"投票后无法看到结果计数"），走 issue→REQ→...→merge，并用 retro 真实修改某个 tunan-\* skill

## 前置

- 在仓库根跑 `./check-tunan-windows.ps1` (Windows) 或 `./check-tunan.sh` (macOS/Linux) 输出 **All checks passed**
  - 校验内容：前置 CLI (git / node / uv / gh / claude / ruflo)、3 个 MCP (memory / context7 / playwright)、tunan-\* skill 集、`.tunan-workspace/{worktrees,retro,raws,sprints}` + `worktrees/.gitignore` + 带 `current_sprint` 的 `settings.md`、`raws/<current>/` 与 `sprints/<current>/reqs/`、`USER.md`、`tunan-tutorials/`、gh 已登录
  - 任一红色 [ERROR] 都先解决再继续
- 当前 git user 即是 sponsor 的 owner 名

随时迷路 → `/tunan-prime` 拉回现状；想用 UI 看池 → `/tunan-dashboard`。

## 命名约定（先看这个）

tunan 全部产物用统一 ID 前缀 `S<MM>-<STAGE><NNN>`：

- `S<MM>` = sprint 号，2 位（`S01`、`S02` ...），当前 sprint 由 `.tunan-workspace/settings.md` 的 `current_sprint` 决定
- `<STAGE>` ∈ `RAW / REQ / PRD / STORY / PLAN / TESTPLAN / PR`
- `<NNN>` = 该 sprint 内该环节 3 位零填充序号（`001`、`002` ...），自动递增，新 sprint 从 `001` 重起
- 示例：`S01-RAW001-voting-system`、`S01-REQ001-voting-system`、`S01-STORY003-see-results`

目录布局（每个 raw / REQ / PRD / STORY 都是自包含子目录，目录名 = 同名 MD 文件名）：

```
.tunan-workspace/
├─ settings.md                # frontmatter 含 current_sprint
├─ worktrees/                 # git worktree 隔离区（.gitignore 屏蔽内容）
├─ retro/                     # retro 历史
├─ raws/                       # raw 收件箱
│   └─ S01/
│       └─ S01-RAW001-voting-system/
│           └─ S01-RAW001-voting-system.md
│           └─ <asset>.png      # 粘贴的截图/视频，语义化命名，同目录平级
└─ sprints/
    └─ S01/
        └─ reqs/
            └─ S01-REQ001-voting-system/
                └─ S01-REQ001-voting-system.md
            └─ S01-PRD001-voting-system/
                └─ S01-PRD001-voting-system.md
            └─ S01-STORY001-create-poll/
                └─ S01-STORY001-create-poll.md
                └─ S01-PLAN001.md
                └─ S01-TESTPLAN001.md
            └─ S01-PR001.md
```

raws/<sprint>/ 与 sprints/<sprint>/reqs/ 按 sprint 同构，可 grep / git blame。

---

## skill 全景 (28 个)

---

按职责分四组：

| 组            | skill                                                                                                                                                                                           | 角色                                                                                       |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| 主链路<br>6 站   | tunan-req / tunan-prd / tunan-story / tunan-plan / tunan-testplan / tunan-dev                                                                                                                   | 把 raw MD 文件逐站推到代码                                                                        |
| PR 闭环        | tunan-pr / tunan-review / tunan-test / tunan-tdd / tunan-merge / tunan-verify                                                                                                                   | PR 自驱动 + 合并后验收                                                                           |
| 入口 / 接管 / 失败 | tunan-triage / tunan-diagnose / tunan-pr-resolve / tunan-takeover / tunan-grill                                                                                                                 | 批量分诊 issue / bug 诊断 / PR 卡死 / 接手他人 / 追问模糊                                                |
| 横切元工具        | tunan-prime / tunan-align / tunan-cp / tunan-parallel-pick / tunan-story-graph / tunan-retro / tunan-improve-arch / tunan-newraw / tunan-raw-check / tunan-dashboard /<br><b>tunan-pipeline</b> | 看现状 / 对齐 / 提交推送 / 并行拣选 / 依赖图 / 回顾 / 架构反思 / 对话起草 raw / 未入池 raw 盘点 / 池可视化看板 / <b>一键全流程</b> |

此外 `.claude/skills/tunan-multi-user/tunan-multi-user.md` 不是可调用的 skill，而是多用户身份字段（`creator/owner/assignee`、`actor_log`）的 SSOT 文档，被各 skill 镜像引用。

**一键全流程：**信任默认时，先用 `/tunan-newraw` 在 `.tunan-workspace/rows/<current_sprint>/` 下落一份 raw，再 `/tunan-pipeline <raw.md>`，它会把下面的步骤 1-9 串起来，每个对齐点应用 ★ 默认。本教程仍按"分站手动"走，便于看清每环职责。

skill / 开关覆盖速查

这张表是本教程的覆盖检查清单：**主线会实际跑常用路径；不适合在投票例子里真实演示的开关，也在这里说明用途和风险边界。**

| skill             | 本教程覆盖方式            | 调用 / 开关说明                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-------------------|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| tunan-req         | 步骤 1、1b、10 实跑      | <code>/tunan-req &lt;raw.md&gt;</code> 默认形态, 必须传 <code>.tunan-workspace/raws/&lt;current_sprint&gt;/</code> 下的 MD 文件路径; <code>--from-issue &lt;#&gt;</code> 从 GitHub issue 入池 (issue 正文自动落档到 <code>raws/&lt;current_sprint&gt;/</code> ); <code>--from-conversation</code> 显式从当前会话抽取 (必须显式指定, 不再缺省); 默认开启 <code>research</code> , <code>--no-research</code> 关闭调研; <code>--kind=feature bug chore</code> 指定类型; <code>--id=S&lt;MM&gt;-REQNNN</code> 指定 id; <code>--owner=&lt;name&gt;</code> 指定 owner。想"一路推到 verify" 走 <code>/tunan-pipeline &lt;raw.md&gt;</code> 。 |
| tunan-prd         | 步骤 2、11-15 实跑      | <code>/tunan-prd pop-next</code> ; <code>/tunan-prd &lt;REQ-id&gt;</code> 指定 REQ; <code>list [--all-owners] [--status=...]</code> 看 PRD 池; <code>show/block/unblock</code> 维护条目。                                                                                                                                                                                                                                                                                                                                                                                     |
| tunan-story       | 步骤 3、11-15 实跑      | <code>/tunan-story pop-next</code> ; <code>/tunan-story &lt;PRD-id&gt;</code> 指定 PRD; <code>list/show/block/unblock</code> 维护 STORY 池。                                                                                                                                                                                                                                                                                                                                                                                                                               |
| tunan-story-graph | 步骤 3 实跑默认 ASCII    | <code>/tunan-story-graph &lt;PRD-id&gt;</code> 输出 ASCII; <code>--mermaid</code> 输出 Mermaid; <code>--all-owners</code> 不限 owner。纯只读, 只画 STORY 粒度。                                                                                                                                                                                                                                                                                                                                                                                                                     |
| tunan-plan        | 步骤 4、11-15 实跑      | <code>/tunan-plan pop-next</code> ; <code>/tunan-plan &lt;STORY-id&gt;</code> 指定 STORY; <code>list/show/block/unblock</code> 维护 PLAN 池。关键决策通过内调 <code>tunan-align</code> 。                                                                                                                                                                                                                                                                                                                                                                                           |
| tunan-testplan    | 步骤 5、11-15 实跑      | <code>/tunan-testplan pop-next</code> ; <code>/tunan-testplan &lt;PLAN-id&gt;</code> 指定 PLAN; <code>list/show/block/unblock</code> 维护 TESTPLAN 池。测试粒度/环境通过 <code>tunan-align</code> 决议。                                                                                                                                                                                                                                                                                                                                                                              |
| tunan-dev         | 步骤 6、11-15 实跑      | <code>/tunan-dev pop-next</code> ; <code>/tunan-dev &lt;TESTPLAN-id&gt;</code> 指定 TESTPLAN; <code>--no-tdd</code> 跳过 red-gate (不推荐, PR body 必须写原因); <code>--resume</code> 续做已有 worktree; <code>--branch=&lt;name&gt;</code> 自定义分支 (默认 <code>tunan/dev/&lt;STORY-id&gt;-&lt;owner&gt;</code> ); <code>--base=&lt;branch&gt;</code> 指定 PR base (默认 <code>dev</code> )。                                                                                                                                                                                                 |
| tunan-tdd         | 步骤 6、10b 间接实跑      | <code>/tunan-tdd &lt;TESTPLAN-id&gt;</code> 直接跑 red-gate; 常由 <code>tunan-dev</code> / <code>tunan-diagnose</code> 内调。                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| tunan-pr          | 步骤 7、11-15 实跑      | <code>/tunan-pr pop-next</code> ; <code>/tunan-pr &lt;PR-id&gt;</code> 跑一轮闭环; <code>--watch [&lt;秒&gt;]</code> 前台轮询, 默认 60 秒且不建议低于 30 秒; <code>list/show/block/unblock</code> 维护 PR 池。                                                                                                                                                                                                                                                                                                                                                                               |
| tunan-review      | 步骤 7 由 tunan-pr 内调 | <code>/tunan-review &lt;PR-id&gt;</code> 可直接触发审查。输出 verdict: <code>pass</code> 或 <code>changes_requested</code> 。                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

| skill            | 本教程覆盖方式                               | 调用 / 开关说明                                                                                                                                                                                     |
|------------------|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| tunan-test       | 步骤 7 由 tunan-pr / 步骤 6 由 tunan-dev 内调 | /tunan-test <PR-id> 或 /tunan-test <TESTPLAN-id> 直接执行 TESTPLAN。<br>tunan-verify 会以内调形式按 TESTPLAN 跑合并后验证。                                                                                       |
| tunan-merge      | 步骤 8 自动衔接                             | /tunan-merge <PR-id> ; --strategy=<merge squash rebase> 选择合并策略，默认 merge ; --no-verify 合后不自动跑 verify（不推荐，需手动 /tunan-verify）。                                                                   |
| tunan-verify     | 步骤 9、11-15 自动衔接                       | /tunan-verify <PR-id> ; --skip-regression 只跑当前 TESTPLAN 三档，不跑最近 merged PR 的额外回归（默认不跳过）。                                                                                                       |
| tunan-triage     | 步骤 10a 可选实跑                           | /tunan-triage 拉所有 open issue; --label=bug 按 label 过滤; --since=<YYYY-MM-DD> 按时间窗过滤; --from-file=<path> 从本地反馈文件批量分诊。                                                                            |
| tunan-diagnose   | 步骤 10b 实跑                             | /tunan-diagnose <REQ-id> 诊断 bug REQ; --from-issue <#> 直接从 issue 诊断; --quick 跳过 align 先快速列假说（仅探索，不替代正式修复链路）。                                                                                   |
| tunan-pr-resolve | 调试备忘说明                                | /tunan-pr-resolve <PR-id> 处理 failed / 冲突 / CI 红; --rebase 显式 rebase 到 base; --abandon 放弃 PR 并清理 worktree/分支，属于高风险出口。                                                                          |
| tunan-takeover   | 调试备忘说明                                | /tunan-takeover <id> 接管为当前 git user; --to=<owner> 改派给他人; --reason=... 写接管理由; --keep-assignee 只改 owner 不动 assignee; --assign-only 只改 assignee。强制接管 in_progress 条目时理由必填。                        |
| tunan-grill      | 步骤 10b、调试备忘说明                         | /tunan-grill <主题 artifact-id> 或 /tunan-grill -- <模糊陈述>。用于把开放模糊问题拆成一次一个、2-3 选 + 默认的可决策问题。                                                                                                      |
| tunan-align      | 全流程内调，步骤 1-5 展示                       | /tunan-align <主题> 可直接调用；更多时候由其他 skill 内调。必须支持 全部默认 / accept all / Q1=B / 自由文本等回应格式。                                                                                                           |
| tunan-prime      | 步骤 0、收尾、调试备忘实跑                        | /tunan-prime 纯只读看现状; --all-owners 看全 owner; --by=<creator assignee owner> 按身份字段聚合（默认 assignee）; --retro=N 展示最近 N 条 retro（默认 3）; --verbose 展开 blocked_by/source_id/priority; --id=<X> 只追踪某条链路。 |
| tunan-cp         | 步骤 1、16、调试备忘说明                        | /tunan-cp [开关] [-- <message>]; --no-push 只 commit; --files=<glob, ...> 限定文件; --message=<text> 或 -- <text> 指定消息; --allow-main 允许主干提交前确认。不接受 --amend/--no-verify/--force。                       |
| tunan-parallel-  | 步骤 3 实跑默认建                            | /tunan-parallel-pick 默认 STORY 池只读建议; --pool=plan 换池; --max=N 限数量; --owner=any 不限 owner; --queue [--max=N] 顺序自动跑一批; --queue                                                                    |

| skill                           | 本教程覆盖方式     | 调用 / 开关说明                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|---------------------------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| pick                            | 议           | <code>--resume</code> 续做中断队列。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <code>tunan-retro</code>        | 步骤 16 实跑    | <code>/tunan-retro</code> 通用复盘; <code>/tunan-retro &lt;id&gt;</code> 围绕事件 (如 <code>S&lt;MM&gt;-REQNNN</code> ) ; <code>--since=&lt;YYYY-MM-DD&gt;</code> 按时间窗复盘。改 skill 必须展示 diff 后二次确认。                                                                                                                                                                                                                                                                                                                                                   |
| <code>tunan-improve-arch</code> | 进阶段落说明      | <code>/tunan-improve-arch</code> 通用架构扫描; <code>--area=&lt;path&gt;</code> 限定子树; <code>--from-retros</code> 只基于 retro 历史推导。只产出新 REQ, 不当场改代码。                                                                                                                                                                                                                                                                                                                                                                                                |
| <code>tunan-newraw</code>       | 步骤 1 实跑     | <code>/tunan-newraw</code> 默认从当前对话抽取, 落成自包含 raw 子目录 <code>.tunan-workspace/rows/&lt;current_sprint&gt;/S&lt;MM&gt;-RAW&lt;NNN&gt;-&lt;slug&gt;/</code> , 内含 <code>&lt;同名&gt;.md</code> + 对话里粘贴的所有 asset (图 / 视频 / log), asset 语义化重命名、MD 同目录平级引用。 <code>&lt;slug&gt;</code> 指定 slug 跳过自动命名; <code>--sprint=S&lt;MM&gt;</code> 指定 sprint; <code>--dry-run</code> 只看计划不动盘; <code>--align</code> 启用逐点对齐 (默认快通路、不弹菜单)。 <code>kind=bug</code> 且 <code>--align</code> 时会做复现完整性自检。产物可直接喂给 <code>/tunan-req</code> 或 <code>/tunan-pipeline</code> 。 |
| <code>tunan-raw-check</code>    | 步骤 1 预备说明   | <code>/tunan-raw-check</code> 扫 <code>rows/&lt;current_sprint&gt;/</code> 与 <code>sprints/&lt;current_sprint&gt;/reqs/S*-REQ*/</code> 做 set diff, 列出尚未被任何 REQ 引用的 raw; <code>--all-sprints</code> 跨 sprint; <code>--sprint=S&lt;MM&gt;</code> 指定 sprint; <code>--show-linked</code> 连已入池的也列; <code>--json</code> 输出 JSON。纯只读, 不动盘。                                                                                                                                                                                                         |
| <code>tunan-dashboard</code>    | 进阶段落说明      | <code>/tunan-dashboard</code> 起本地 Python http 看板 (默认 <code>port=37856</code> , 自动开浏览器), 7 列展示 raw→REQ→PRD→STORY→PLAN→TESTPLAN→PR, 可在 UI 直接改 <code>assignee/owner/status/priority</code> 并自动追写 <code>actor_log</code> ; <code>--port=&lt;N&gt;</code> 换端口; <code>--workspace=&lt;path&gt;</code> 指定 workspace; <code>--no-open</code> 不自动开浏览器; <code>--read-only</code> 只读模式; <code>--stop</code> 停掉看板。                                                                                                                                     |
| <code>tunan-pipeline</code>     | 全景说明、调试备忘说明 | <code>/tunan-pipeline &lt;raw.md&gt;</code> 默认形态, 从 raw MD 文件全链路; <code>--from-issue &lt;#&gt;</code> 从 issue; <code>--from-conversation</code> 显式从当前对话抽取; <code>&lt;REQ-id&gt; / &lt;PRD-id&gt; / &lt;STORY-id&gt;</code> 从已有条目续推; <code>--stop-at=plan</code> 到 PLAN 停; <code>--no-merge</code> 到 <code>sponsor_wait</code> 停; 默认 H1 red-gate 与 H2 PR LGTM 预授权一路到底, <code>--need-auth</code> 才回到逐 gate 等待 sponsor。                                                                                                                       |

## 第一轮 · 黄金路径

### 步骤 0 · prime 看初始状态

```
sponsor> /tunan-prime
```

预期 claude 输出: 池全空、无 worktree、无 PR、retro 空。推荐下一步 `/tunan-newraw` 或 `/tunan-req`。

## 步骤 1 • 起草 raw → 生成 REQ (缺省 --research)

最省事的方式：在对话里把需求说清楚（可以贴截图），然后让 `/tunan-newraw` 把它落成 raw 子目录：

```
sponsor> 我想做一个投票系统：可以投票去哪里旅游 / 中饭吃什么。  
          （随便展开几句细节：截止时间？单选/多选？匿名/实名？想不清也没关系。）  
  
sponsor> /tunan-newraw
```

预期：claude 把对话里的需求陈述 + 所有粘贴的 asset 一步落成自包含子目录 `.tunan-workspace/raWS/S01/S01-RAW001-voting-system/`，内含 `S01-RAW001-voting-system.md`（H1 下标注 `> **类型** / > **创建者**`）+ 语义化重命名的 asset。缺省走快通路、不弹对齐菜单；要逐点确认传 `--align`。

想自己手写 raw 也行——按命名约定建子目录即可：

```
sponsor> mkdir -p .tunan-workspace/raWS/S01/S01-RAW001-voting-system  
sponsor> # 在 S01-RAW001-voting-system/ 下写 S01-RAW001-voting-system.md
```

raw 落盘后入池：

```
sponsor> /tunan-req .tunan-workspace/raWS/S01/S01-RAW001-voting-system/S01-RAW001-voting-s
```

真的只想从对话原话抽取、不落 raw 草稿？给 `/tunan-req` 显式传 `--from-conversation`；默认不再静默 fallback 到对话。

写了一批 raw、想知道哪些还没入 REQ 池？跑 `/tunan-raw-check` ——它对 `raWS/` 与 `sprints/ ... /reqs/` 做 set diff，列出未被任何 REQ 引用的 raw。

预期：

- claude **开始**播报"将走 1) 抽取 2) research 3) top3 4) 入池"
- 抽取意图后，用 **WebSearch / context7 跑社区调研**：找类似投票工具的常见复杂度（截止时间、多选 vs 单选、防刷票、匿名/实名、结果可见性）
- 把发现进 Open Questions（不直接放大 scope）
- 内调 `tunan-align` 提 ≤6 个 Q（每个 top3+★默认）：
  - Q1 单选 / 多选（★单选）
  - Q2 截止时间精度（★分钟）
  - Q3 防刷票（★每用户一票）

- Q4 匿名 (★实名展示)
- Q5 结果可见性 (★截止前可见)
- Q6 优先级 (★P2)

```
sponsor> 全部默认
```

预期:

- 写入 `.tunan-workspace/sprints/S01/reqs/S01-REQ001-voting-system/S01-REQ001-voting-system.md`, `status: ready` (sprint 由 `settings.md.current_sprint` 决定)
- 结束播报: 推荐 `/tunan-prd S01-REQ001` / `/tunan-cp` / `/tunan-prime`

### 步骤 1b • (可选) 从 issue 生成 REQ

如果你不想自己写 raw, 可以先在 GitHub 开 issue (issue 正文会自动落档一份到 `.tunan-workspace/raws/<current_sprint>/`) :

```
sponsor> gh issue create --title "投票系统: 吃饭/旅游目的地" --body "... "
sponsor> /tunan-req --from-issue 1
```

预期: claude 调 `gh issue view 1`, 把 title/body/讨论作为输入, 其余流程同步骤 1。

### 步骤 2 • REQ → PRD

```
sponsor> /tunan-prd S01-REQ001
```

预期:

- 弹 S01-REQ001, 置 `in_progress`
- 起草 PRD: 把 6 个决议映射到 FR; 提 a11y / 性能 NFR (research 发现 + sponsor 没说的进 Open Questions)
- 内调 align: Persona 取舍 / NFR 阈值 / 是否纳入"防刷票更强机制"

```
sponsor> 全部默认
```

预期: 写入 `<REQ-dir>/S01-PRD001-voting-system/S01-PRD001-voting-system.md`, `status=ready`; 推荐 `/tunan-story S01-PRD001`。



### 步骤 3 • PRD → STORY 拆分

```
sponsor> /tunan-story S01-PRD001
```

预期：

- claude 提 5 个 STORY 候选（标准切分）：
  1. S01-STORY001：创建投票（标题、选项、单/多选、截止时间）
  2. S01-STORY002：投一票
  3. S01-STORY003：看结果（计数、百分比）
  4. S01-STORY004：截止后只读
  5. S01-STORY005：每用户一票约束
- 内调 align：
  - Q1 切分粒度（★标准 5 个 / 粗 3 个 / 细 8 个）
  - Q2 哪些 P0（★STORY001/002/003）
  - Q3 STORY004/005 是否本轮做（★做）

```
sponsor> 全部默认
```

预期：5 个 `<PRD-dir>/S01-STORYNNN-*/S01-STORYNNN-*.md` 落盘，`source_id: S01-PRD001`，依赖图：

- STORY002 blocked\_by STORY001
- STORY003 blocked\_by STORY002
- STORY004/005 blocked\_by STORY002

```
sponsor> /tunan-story-graph S01-PRD001
```

预期：ASCII 树状依赖图（可加 `--mermaid` 贴 GitHub）。看清哪些 STORY 已 done、哪些可以**并行**起：

```
sponsor> /tunan-parallel-pick
```

预期：claude 按 blocked\_by + 影响文件冲突分析，建议本轮 S01-STORY001 单跑（根节点）；STORY004/005 解锁后可与 STORY003 并行。

### 步骤 4 • STORY → PLAN

从根节点 S01-STORY001 开始：

```
sponsor> /tunan-plan S01-STORY001
```

预期:

- claude 用 Glob/Grep 探现有代码 (这是教程, 假设是个新项目; 输出"未找到现有相关代码, 按 greenfield 处理")
- 起草 PLAN: 技术栈选型 / Affected Files (新建) / Steps / Risks / Rollback
- align:
  - Q1 框架选型 (★Next.js + SQLite / Express+React / FastAPI+React)
  - Q2 是否用 feature flag (★否, 本期纯新建)
  - Q3 测试覆盖 (★单测 + e2e 关键路径)

```
sponsor> 全部默认
```

落盘 <STORY-dir>/S01-PLAN001.md 。

重复 /tunan-plan for S01-STORY002..005 或 /tunan-parallel-pick 取并行项。

## 步骤 5 • PLAN → TESTPLAN

```
sponsor> /tunan-testplan S01-PLAN001
```

预期:

- 从 S01-STORY001 AC 出测试用例
- 从 PLAN Affected Files 出回归 (greenfield 阶段回归集为空)
- Persona: novice 创建投票 / power-user 复用模板 / adversarial 标题超长 / 选项 0 个 / 截止时间在过去

```
sponsor> 全部默认
```

落盘 <STORY-dir>/S01-TESTPLAN001.md 。

## 步骤 6 • TESTPLAN → worktree dev

```
sponsor> /tunan-dev S01-TESTPLAN001
```

预期:

- claude 检查依赖 (S01-PLAN001 + S01-STORY001 ready)
- 创建 worktree: `git worktree add .tunan-workspace/worktrees/S01-STORY001-jonli tunan/dev/S01-STORY001-jonli`
- 进入 TDD 模式 ( `tunan-tdd` 缺省强制 red-gate, 见 memory feedback\_tdd\_red\_gate)
  - 先写 TESTPLAN 中的失败测试 → 跑 → 红
  - sponsor 显式放行才写产品代码 → 绿
- 完成后 `git push` 起 PR: `gh pr create --base dev`
- 落盘 `<STORY-dir>/S01-PR001.md`, `gh_pr: <num>`, `status: reviewing`

## 步骤 7 • PR 自驱动循环

```
sponsor> /tunan-pr S01-PR001
```

预期一次调用:

1. fetch 评论 → 无
2. reviewer 跑一轮 → 发现"缺少输入校验" → 写 PR 评论
3. coder 修 → push → reviewer 再跑 → pass
4. tester 跑 → smoke pass / 1 个 adversarial 失败"标题 1024 字符崩溃" → 写评论
5. coder 修 → push → tester 再跑 → 全 pass
6. 进入 `sponsor_wait`, 播报"等 sponsor approve; 可 `/tunan-pr S01-PR001 --watch` 持续轮询"

```
sponsor> (在 GitHub PR 上评论 LGTM)
```

```
sponsor> /tunan-pr S01-PR001
```

预期: 检测到 sponsor approve → 自动调 `/tunan-merge S01-PR001`。

## 步骤 8 • merge

(自动衔接, 无需手动)

预期:

- `gh pr merge <num> --merge` → 合到 dev 分支
- 删 worktree + 分支
- PR 池 status=merged
- 自动调 `/tunan-verify S01-PR001`

## 步骤 9 • verify

预期：

- checkout dev
- 跑 S01-TESTPLAN001 全部用例 → 全过
- 链路状态级联：S01-PLAN001/S01-STORY001 done；S01-PRD001/S01-REQ001 仍 in\_progress（等其余 STORY 走完）
- 结束播报：推荐 `/tunan-plan S01-STORY002`（依赖根已解锁）

重复步骤 4-9 走 S01-STORY002..005，黄金路径完结。

## 第一轮收尾

```
sponsor> /tunan-prime
```

预期：所有池条目都 `done`；retro 空；推荐"开始第二轮迭代"。

## 第二轮 • issue 入口 + retro 进化

### 预设 bug

合并后，sponsor 自己用一下投票系统，发现：**投了一票后页面没立即显示新计数**（需手动刷新）。开 GitHub issue：

```
sponsor> gh issue create --title "投票后无法看到结果计数" --body " ... "
```

### 步骤 10a • （可选）批量分诊

如果一次性堆了 N 个 issue，先批量分诊去重、分类、路由：

```
sponsor> /tunan-triage
```

预期：claude 拉所有 open issue，逐条 top3 分类（→REQ / 合并到既有 / 丢弃），收集 sponsor 一次确认后批量入 REQ 池。

## 步骤 10 • issue → REQ (kind: bug)

```
sponsor> /tunan-req --from-issue 2
```

预期：

- 抽取 issue
- 推断 kind=bug (USER.md 决策风格 → 给 top3 让 sponsor 确认; ★ bug)
- research 关 / 开均可 (bug 的 research 通常关)
- align: Q1 优先级 (★P1, 影响主流程) / Q2 是否本期修 (★是)

```
sponsor> 全部默认
```

落盘 `sprints/S01/reqs/S01-REQ003-vote-count-not-refresh/S01-REQ003-vote-count-not-refresh.md` , `kind: bug` , `status: ready` 。

## 步骤 10b • bug 先诊断再修

bug 流程比 feature 多一站：先诊断根因，再写 PRD。

```
sponsor> /tunan-diagnose S01-REQ003
```

预期：

- 收集现象 (复现步骤 / 期望 / 实际 / 环境) ; 缺则内调 `tunan-grill` 追问
- 调 `tunan-tdd` 写一个只复现该 bug 的失败测试 (红)
- 列根因假说 top3 + ★默认; 不当场修
- 产物: 诊断报告 (贴在 REQ 评论里) , 状态可流转 → `/tunan-prd S01-REQ003` 走正常修复流程

## 步骤 11-15 • 走完整流程 (同第一轮, 从 PRD 到 verify)

bug 流程的 PRD/STORY 通常短小 ("Given 用户提交投票 When 提交成功 Then 计数立即更新无需刷新") , PLAN 里直接引用诊断报告里的"根因 + 复现测试"。

## 步骤 16 • retro: 让流程进化

```
sponsor> /tunan-retro S01-REQ003
```

预期 claude 提问：

"issue S01-REQ003 (投票后无法看到结果计数) 出现了。我们的 rules / skills / workflow 中有什么可以改的, 让这种问题不再发生? "

claude 通过 align 提候选改进 (top3+★默认) :

- **C-1:** 在 `tunan-testplan` 的 persona 模板里强制加 "提交后立即可见" 类用例
  - ★ 默认。最便宜, 规范化未来 TESTPLAN
- **C-2:** 在 `tunan-review` 增加"前后端状态同步"检查项
  - 备选。范围更大但更彻底
- **C-3:** 不改 skill, 加一条 retro 仅记录
  - 最弱

```
sponsor> 默认
```

预期:

- claude 展示对 `.claude/skills/tunan-testplan/SKILL.md` 的具体改动 diff
- **必须 sponsor 二次确认才落盘** (防 skill 自毁)

```
sponsor> 落盘
```

预期:

- 改写 `tunan-testplan SKILL.md` (在 Persona Scenarios 模板里加一行"提交后即时反馈")
- 写 `retro/2026-MM-DD-vote-count-not-refresh.md` (含触发 issue id / 改了哪些 skill / 决议来源)
- 推荐 `/tunan-cp` 提交 retro 改动

## 第二轮收尾

至此你已经:

1. 走通 feature 全流程
2. 走通 bug 全流程 (含 `tunan-diagnose` 前置诊断)
3. 让 retro **真实地**修改了 tunan 自己 — 这就是"skills 可进化"的含义

## 进阶 • 池可视化与架构反思

任何时候想用 UI 看池 (而不是读 `/tunan-prime` 文本), 起看板:

```
sponsor> /tunan-dashboard
```

预期: claude 起一个本地 Python http 服务 (默认端口 37856) 并开浏览器, 7 列从左到右展示 raw→REQ→PRD→STORY→PLAN→TESTPLAN→PR; 卡片可按 creator/owner/assignee/status/sprint 过滤; 可在 UI 直接改 assignee/owner/status/priority 并自动追写 actor\_log。 --stop 停掉。  
如果 retro 反复指向同一片代码 ("为啥这块改 5 次"), 不要再开单点 REQ, 调:

```
sponsor> /tunan-improve-arch --from-retros
```

预期: claude 扫 retro 历史 + 代码热点, 提出架构改进 top3 (拆模块 / 抽象层 / 测试基建), 产出仍走正常 REQ→PRD→... 流程, 不当场动代码。

## 教程产物清单 (应在 .tunan-workspace/ 中看到的)

```
settings.md # current_sprint: S01
raws/S01/
  S01-RAW001-voting-system/
    S01-RAW001-voting-system.md
sprints/S01/reqs/
  S01-REQ001-voting-system/
    S01-REQ001-voting-system.md (done)
  S01-PRD001- ... /
    S01-PRD001- ... md (done)
  S01-STORY001..005- ... /
    S01-STORYNNN-*.md / S01-PLANNNN.md / S01-TESTPLANNNN.md / S01-PRNNN.md (全 done)
  S01-REQ003-vote-count-not-refresh/ (kind:bug)
    S01-REQ003-vote-count-not-refresh.md (done)
  S01-PRD003- ... /
    S01-PRD003- ... md (done)
    S01-STORY006- ... / (done, bug)
      S01-STORY006-*.md / S01-PLAN/TESTPLAN/PR-*.md
retro/
  2026-MM-DD-vote-count-not-refresh.md
```

skill 改动: .claude/skills/tunan-testplan/SKILL.md 在 Persona 模板加了一行。

## 调试备忘

---

- 任何时候迷路 → `/tunan-prime` ; 想看 UI → `/tunan-dashboard`
  - pop-next 总是空 → 检查 owner (默认仅你自己) ; 试 `--all-owners` ; 或 `/tunan-takeover <id>` 接手他人的条目
  - 信息太含糊连 top3 都凑不齐 → `/tunan-grill <主题|id>` 把模糊拆成可枚举候选
  - worktree 没清掉 → `git worktree list` 看; `git worktree remove <path>`
  - PR 自驱动卡在 reviewing → `/tunan-pr <id>` 再调一次; 或 `--watch`
  - PR 状态 failed / 合并冲突 / CI 红 / 评论挤压不收敛 → `/tunan-pr-resolve <PR-id>` (必要时 `--rebase` 或 `--abandon`)
  - 想中断流程 → 手 `git status` 看; 不要硬 reset, 先看 PR 池条目 status
  - 想一键跑全流程而不是分站 → 先 `/tunan-newraw` 落一份 raw, 再 `/tunan-pipeline <raw.md>` (参 skill 全景表)
  - 不确定哪些 raw 还没入 REQ 池 → `/tunan-raw-check`
- 

## 接下来

---

- 把这套流程套到你的真实项目上
- 关键 skill 全装好后, sponsor 平均决策次数  $\approx$  "阶段数  $\times$  1" (每阶段一句"全部默认"即可)
- retro 用得越勤, 未来流程越省事